Overview of Statistical Learning and Modeling – 36-600

Lab 8T: Logistic Regression
Due Wednesday, October 22, 11:59pm

Trishla Nair

Data

Democracy day is coming up - let's take a look at one aspect of how political movements have shaken out over the years. We'll begin by importing data on political movements (this is a subset of columns from the original NAVCO dataset, which has since grown substantially; see this page).

These data, as processed, contain information on 218 political movements. The predictor variables are largely categorical: `nonviol`, for instance, is `YES` if the movement was non-violent, etc. In particular, `aid` indicates if the government being targeted received foreign aid to deal with the movement, and `defect` indicates whether substantial portions of the military and police sided with the movement. `democracy` ranges from -10 for authoritarian regimes to 10 for fully democratic regimes.

Questions

Question 1

Summarize the data via `summary()`. Which variable looks like it might benefit from a transform to mitigate right-skewness? Create a histogram for that variable, then make the transformation (by, e.g., doing `df$x <- sqrt(df$x)`, where `x` should be replaced with the variable name, and yes, that's `sqrt()`, not `log()`, this time) and create a histogram of the transformed variable.

```
summary(df)
```

```
##  nonviol     democracy        sanctions  aid       support    viol.repress
##  NO :149   Min.   :-10.0000   NO :179   NO :149   NO :159   NO : 27
##  YES: 69   1st Qu.: -7.0000   YES: 39   YES: 69   YES: 59   YES:191
##            Median : -3.0000
##            Mean   : -0.9404
##            3rd Qu.:  7.0000
##            Max.   : 10.0000
##  defect      duration         label
##  NO :157   Min.   :    2   FAILURE:123
##  YES: 61   1st Qu.:  360   SUCCESS: 95
##            Median :  730
##            Mean   : 1494
##            3rd Qu.: 1825
##            Max.   :17155
```

```
df$duration <- sqrt(df$duration)
```

Duration should be transformed.

Question 2

Split the data into training and test sets. Remember to set the seed!

```
set.seed(42)
train_index <- sample(x = 1:nrow(df),size = floor(0.7 * nrow(df)),replace = FALSE)

train_set <- df[train_index, ]
test_set <- df[-train_index, ]
```

Question 3

Carry out a logistic regression analysis (predicting the response variable `df$label`, i.e., the ultimate success/failure of the movement, from all other covariates), and display both the misclassification rate and a table of predictions versus true responses on the test-set (i.e., display the confusion matrix). (Beyond the notes, you might want to look at the code on pp. 156-158 of ISLR, 1st ed.) What is your misclassification rate? (Save the output of your call to `table()` as `tab` so that we can use it later.)

```
out.log <- glm(label ~ ., data = train_set, family = binomial, control = list(maxit = 100))


resp.prob <- predict(out.log, newdata = test_set, type ="response")
resp.pred <- ifelse(resp.prob > 0.5, "SUCCESS", "FAILURE")
resp.pred <- factor(resp.pred, levels = levels(train_set$label))

y_true <- test_set$label

misclassification_rate <- mean(resp.pred != y_true)
print(paste("Misclassification Rate:", round(misclassification_rate, 4)))
```

```
## [1] "Misclassification Rate: 0.2727"
```

```
# Confusion matrix
tab <- table(Predicted = resp.pred, Actual = y_true)
print("Confusion Matrix:")
```

```
## [1] "Confusion Matrix:"
```

```
print(tab)
```

```
##            Actual
## Predicted FAILURE SUCCESS
##   FAILURE      28       9
##   SUCCESS       9      20
```

```
My misclassification rate is 0.2727.
```

Question 4

What are the class proportions for the (test-set!) response variable? Use these numbers to determine the "null MCR," i.e., the misclassification rate if we simply guess that all data belong to the majority class. Recall that summing the output of logical operations (e.g., `sum(df.test$label == "NO")`) is a concise way to display the counts of "yes" and "no". How does this null rate compare to that found via logistic regression?

```
class_counts <- table(test_set$label)
class_props  <- prop.table(class_counts)

print("Class proportions in test set:")
```

```
## [1] "Class proportions in test set:"
```

```
print(class_props)
```

```
##
##   FAILURE   SUCCESS
## 0.5606061 0.4393939
```

```
majority_class <- names(which.max(class_counts))
null_mcr <- sum(test_set$label != majority_class) / nrow(test_set)

cat("Null model MCR:", null_mcr)
```

```
## Null model MCR: 0.4393939
```

```
pred_prob <- predict(out.log, newdata = test_set, type ="response")

pred_class <- ifelse(pred_prob > 0.5, "YES", "NO")
logistic_mcr <- mean(pred_class != test_set$label)

cat("Logistic regression MCR:", logistic_mcr)
```

```
## Logistic regression MCR: 1
```

```
The null rate is approximately half of the logistic rate.
```

Question 5

While statisticians often shoe-horn results into the facade of test-produced error rates, in many contexts other rates are used. Compute the *sensitivity* and *specificity* of logistic regression using definitions on this web page. There can be some ambiguity regarding labeling results of tables: assume that predicting success for a movement that was successful is a "true positive," while predicting failure for a successful movement is a "false negative," etc.

Don't hard-code any counts/rates! If you saved your confusion matrix above to the variable `tab`, then, e.g.:

```
TP <- tab[2, 2] # second row, second column (i.e., bottom right)
FP <- tab[2, 1] # second row, first column (i.e., bottom left)
                # etc.
```

Map your table to TP, FP, TN, and FN, and use these to compute sensitivity and specificity, and then define each of these in words. In a perfect world, the sum of sensitivity and specificity would be 2.

```
TP <- tab[2, 2]
FP <- tab[2, 1]
FN <- tab[1, 2]
TN <- tab[1, 1]
sensitivity<- TP/(TP+FN)
specififcity<- TN/(TN+FP)
print(sensitivity)
```

```
## [1] 0.6896552
```

```
print(specififcity)
```

```
## [1] 0.7567568
```

Sensitivity measures ability of a model to correctly identify if a protest was nonviolent. Sensitivity measures ability of a model to correctly identify if a protest was violent. ## Question 6

A political scientist might be more interested to know what proportion of movements that are predicted to be successful actually are successful. Compute this quantity and determine from the aforementioned confusion matrix (Wikipedia) page what this quantity is usually called.

```
precision<- TP/(TP+FP)
print(precision)
```

```
## [1] 0.6896552
```

```
This is precision.
```

Question 7

Let's go back to the output from the logistic regression fit to the training data. Pass that output to the `summary()` function here. Look at the output, but before you interpret it, let's review what the output for a categorical predictor variable means. Take `aid`, for instance. The reference level is `NO`, meaning the government did not receive foreign aid specifically to quell the movement... and for that reference level, the coefficient is implicitly zero (and never explicitly shown in the output). For `YES`, the coefficient is (for my data split) -0.148. (Your coefficient may be and probably will be slightly different.) You can think of what this means in terms of relative odds: is foreign aid to the government "under attack" associated with an increase the probability of success, or a decrease? If we compute

$$e^{-0.148} = 0.862\,,$$

we see that, all else being equal, having foreign aid reduces the odds of a movement's success by about 14%, i.e., aid helps governments repress movements, on average. This all being said: identify the variable that is most informative about predicting *successful* movements, and the variable that is most informative about predicting *failed* movements. (And definitely don't include the intercept term here!)

```r
summary(out.log)
```

```
##
## Call:
## glm(formula = label ~ ., family = binomial, data = train_set,
##     control = list(maxit = 100))
##
## Coefficients:
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -0.836440   0.765475  -1.093  0.27452
## nonviolYES       2.277147   0.515851   4.414 1.01e-05 ***
## democracy        0.074569   0.032597   2.288  0.02216 *
## sanctionsYES     0.620074   0.553549   1.120  0.26264
## aidYES           0.195590   0.426351   0.459  0.64641
## supportYES       0.733846   0.555382   1.321  0.18639
## viol.repressYES -1.105202   0.679179  -1.627  0.10368
## defectYES        1.372717   0.478353   2.870  0.00411 **
## duration         0.005154   0.011284   0.457  0.64784
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 208.08  on 151  degrees of freedom
## Residual deviance: 159.75  on 143  degrees of freedom
## AIC: 177.75
##
## Number of Fisher Scoring iterations: 4
```

```r
print(exp(2.277147)) # Odds ratio for nonviolYES
```

```
## [1] 9.748827
```

```r
print(exp(-1.105202)) # Odds ratio for viol.repressYES
```

```
## [1] 0.331144
```

```
The odds ratio for nonviolent protests of 9.748827 shows that they are more likely to succeed. The odds
```

Question 8

Is the logistic regression model *significant*, in a statistical sense? In other words, is at least one of the coefficients in the model truly non-zero? Go back to the summary and see the lines indicating the `Null deviance` and the `Residual deviance`. If you named your output from `glm()` as `log.out`, then you can get the null deviance from `log.out$null.deviance` and the residual deviance from `log.out$deviance`. Similarly, you can get the associated numbers of degrees of freedom from `log.out$df.null` and `log.out$df.residual`. Why would you want to do this? Well, if you took the absolute value of the difference in deviances (call this `dev.diff`) and the difference in degrees of freedom (`df.diff`), you could then do a formal hypothesis test: for a useful model, `dev.diff` should **not** be chi-square distributed with `df.diff` degrees of freedom. In other words, if the p-value `1 - pchisq(dev.diff, df.diff)` is less than 0.05, at least one of the coefficients is truly non-zero. (This is analogous to doing an F-test in a linear regression models; there, the null hypothesis is that all the slopes are zero.) Compute the p-value here. Do you reject the null hypothesis that all the coefficients are truly zero?

```r
print(out.log$null.deviance)
```

```
## [1] 208.0775
```

```r
print(out.log$df.null)
```

```
## [1] 151
```

```r
print(out.log$deviance)
```

```
## [1] 159.7488
```

```r
print(out.log$df.residual)
```

```
## [1] 143
```

```r
dev.diff <- abs(out.log$null.deviance - out.log$deviance)

df.diff <- abs(out.log$df.null - out.log$df.residual)
print(dev.diff)
```

```
## [1] 48.32873
```

```r
print(df.diff)
```

```
## [1] 8
```

```r
p_value <- 1 - pchisq(dev.diff, df.diff)
print(p_value)
```

```
## [1] 8.547988e-08
```

```
Since the p-value is so small, I reject the null hypothesis that all the coefficients are truly zero.
```